Face Detection using Haar Cascades with OpenCV and Matplotlib

Name : M.VIRUMAA HARISH

Reg No : 212223230246

Aim

To write a Python program using OpenCV to perform the following image manipulations:
i) Extract ROI from an image.
ii) Perform face detection using Haar Cascades in static images.
iii) Perform eye detection in images.
iv) Perform face detection with label in real-time video from webcam.

Software Required

- Anaconda - Python 3.7 or above
- OpenCV library (`opencv-python`)
- Matplotlib library (`matplotlib`)
- Jupyter Notebook or any Python IDE (e.g., VS Code, PyCharm)

Algorithm

I) Load and Display Images

- Step 1: Import necessary packages: `numpy` , `cv2` , `matplotlib.pyplot`
- Step 2: Load grayscale images using `cv2.imread()` with flag `0`
- Step 3: Display images using `plt.imshow()` with `cmap='gray'`

II) Load Haar Cascade Classifiers

- Step 1: Load face and eye cascade XML files

III) Perform Face Detection in Images

- Step 1: Define a function `detect_face()` that copies the input image
- Step 2: Use `face_cascade.detectMultiScale()` to detect faces
- Step 3: Draw white rectangles around detected faces with thickness 10
- Step 4: Return the processed image with rectangles

IV) Perform Eye Detection in Images

- Step 1: Define a function `detect_eyes()` that copies the input image

- Step 2: Use `eye_cascade.detectMultiScale()` to detect eyes
- Step 3: Draw white rectangles around detected eyes with thickness 10
- Step 4: Return the processed image with rectangles

V) Display Detection Results on Images

- Step 1: Call `detect_face()` or `detect_eyes()` on loaded images
- Step 2: Use `plt.imshow()` with `cmap='gray'` to display images with detected regions highlighted

VI) Perform Face Detection on Real-Time Webcam Video

- Step 1: Capture video from webcam using `cv2.VideoCapture(0)`
- Step 2: Loop to continuously read frames from webcam
- Step 3: Apply `detect_face()` function on each frame
- Step 4: Display the video frame with rectangles around detected faces
- Step 5: Exit loop and close windows when ESC key (key code 27) is pressed
- Step 6: Release video capture and destroy all OpenCV windows

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Step 1: Read the image and convert the image into RGB
image = cv2.imread('naveen.jpeg')  # Replace with your image path
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Step 2: Display the original image
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('on')
plt.show()
# Step 4: Set the pixels to display the ROI (Region of Interest)
# Define the coordinates for the Region of Interest (ROI)
# (startY:endY, startX:endX)
roi = image[100:420, 200:550]  # ROI coordinates (adjust as needed)

# Create a blank mask of the same size as the original image
mask = np.zeros_like(image)

# Place the ROI on the mask
mask[100:420, 200:550] = roi
# Step 5: Perform bitwise conjunction of the two arrays using bitwise_and
segmented_roi = cv2.bitwise_and(image, mask)
# Step 6: Display the segmented ROI from the image
segmented_roi_rgb = cv2.cvtColor(segmented_roi, cv2.COLOR_BGR2RGB)
plt.imshow(segmented_roi_rgb)
plt.title("Segmented ROI")
plt.axis('off')
plt.show()
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Step 1: Read the image and convert it to RGB for displaying
image = cv2.imread('dipt.jpeg')  # Replace with your actual image file path
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  # Convert BGR to RGB
# Original Image
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
# Step 2: Convert the image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  # Convert to grayscale
# Step 3: Apply Gaussian blur to reduce noise
blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 0)  # Apply Gaussian blur (5x5 kernel)
# Step 5: Use Canny edge detector to find edges
edges = cv2.Canny(blurred_image, 50, 150)  # Detect edges using Canny (thresholds 50 and 150)
# Canny Edge Detection
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
# Step 6: Find contours in the edged image
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# Step 7: Filter contours based on area and draw bounding boxes
result_image = image.copy()  # Create a copy of the original image to draw bounding boxes
for contour in contours:
    if cv2.contourArea(contour) > 50:  # Filter out small areas
        x, y, w, h = cv2.boundingRect(contour)  # Get the bounding box for the contour
        cv2.rectangle(result_image, (x, y), (x + w, y + h), (0, 255, 0), 2)  # Draw the
rectangle
# Handwriting Detection Result
plt.imshow(cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB))
plt.title("Handwriting Detection")
plt.axis('off')
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Step 1: Set and add the config_file, weights to your folder
# Ensure you have the MobileNet-SSD files downloaded:
# Download: https://github.com/chuanqi305/MobileNet-SSD
config_file = 'deploy.prototxt'  # Path to the config file
weight
s = 'mobilenet_iter_73000.caffemodel'  # Path to the weights file
# Step 2: Use a pretrained DNN model (MobileNet-SSD v3)
net = cv2.dnn.readNetFromCaffe(config_file, weights)
# Step 4: Create a class label and print the same
class_labels = {0: 'background', 1: 'aeroplane', 2: 'bicycle', 3: 'bird', 4: 'boat',
                5: 'bottle', 6: 'bus', 7: 'car', 8: 'cat', 9: 'chair', 10: 'cow', 11:
'diningtable',
                12: 'dog', 13: 'horse', 14: 'motorbike', 15: 'person', 16: 'pottedplant', 17:
'sheep',
                18: 'sofa', 19: 'train', 20: 'tvmonitor'}
# Step 5: Read the image
image = cv2.imread('itac.jpeg')  # Replace with your image path
(h, w) = image.shape[:2]
# Convert image to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Create a blob for DNN processing
blob = cv2.dnn.blobFromImage(image, 0.007843, (300, 300), 127.5)
# Step 6: Set the model and threshold to 0.5
```

```
net.setInput(blob)
detections = net.forward()
# Step 7: Flatten the index, confidence
for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]

    if confidence > 0.5:  # Confidence threshold
        index = int(detections[0, 0, i, 1])  # Get class index
        label = class_labels[index]  # Get label name
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = box.astype("int")
# Step 8: Draw rectangles and labels on the image
        cv2.rectangle(image_rgb, (startX, startY), (endX, endY), (0, 255, 0), 2)
        cv2.putText(image_rgb, label, (startX, startY - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(255, 0, 0), 2)
# Step 9: Display the image using Matplotlib
plt.imshow(image_rgb)
plt.title("Object Detection with MobileNet-SSD")
plt.axis("off")
plt.show()
```

OUTPUT

Original Image

Segmented ROI

Original Image

Canny Edge Detection

Canny Edge Detection

Handwriting Detection

Handwriting Detection

Object Detection with MobileNet-SSD

Object Detection with MobileNet-SSD

RESULT

Thus to write a Python program using OpenCV to perform the following image manipulations was verified successfully.